# Homework 1: Bayesian linear regression

STAT 348, UChicago, Spring 2026

---

**Your name here: Oleg Glotov**

**Hours spent: 16**

(Please let us know how many hours in total you spent on this assignment so we can calibrate for future assignments. Your feedback is always welcome!)

---

Open in Colab

---

## Instructions

This homework focuses on themes in the first three lectures and will also get you familiar with Python and PyTorch which we will use for the rest of the course.

For reference, this homework is a close adaption of HW1 for Scott Linderman's STATS 305C, for which the slides for lecture 1 may be a useful reference.

Assignment is due **Sunday April 5, 11:59pm** on GradeScope.

In [1]:
```
import torch
from torch.distributions import Normal, Gamma, \
    TransformedDistribution, MultivariateNormal
from torch.distributions.transforms import PowerTransform

import matplotlib.pyplot as plt
import seaborn as sns
sns.set_context("notebook")
```

## Bayesian Linear Regression

Let $\{\mathbf{x}_i, y_i\}_{i=1}^n$ denote a dataset with covariates $\mathbf{x}_i \in \mathbb{R}^p$ and scalar outcomes $y_i \in \mathbb{R}$. Let $\mathbf{X} \in \mathbb{R}^{n \times p}$ denote the design matrix where each row is a vector of covariates and $\mathbf{y} \in \mathbb{R}^n$ denote the vector of outcomes.

We will model the outcomes as conditionally independent Gaussian random variables given the covariates and the parameters,

$$p(\mathbf{y} \mid \boldsymbol{\beta}, \sigma^2, \mathbf{X}) = \prod_{i=1}^{N} \mathcal{N}(y_i \mid \mathbf{x}_i^\top \boldsymbol{\beta},\, \sigma^2),$$

where $\boldsymbol{\beta} \in \mathbb{R}^p$ are the *regression coefficients* and $\sigma^2 \in \mathbb{R}_+$ is the *conditional variance*.

As discussed in lecture 2, in *Bayesian* linear regression we place a priors over the parameters. In lecture, we placed a simple multivariate Gaussian prior over the coefficients $\boldsymbol{\beta}$ and treated the variance $\sigma^2$ as a fixed and known hyperparameter. In this homework, we will place a *joint prior over both* parameters. We first place a *scaled inverse chi-squared* prior over $\sigma^2$:

$$\begin{aligned} P(\sigma^2 \mid v_0, \tau_0^2) &= \chi^{-2}(\sigma^2;\, v_0, \tau_0^2) \\ &= \frac{(\frac{\nu_0 \tau_0^2}{2})^{\frac{\nu_0}{2}}}{\Gamma(\frac{\nu_0}{2})} (\sigma^2)^{-\frac{\nu_0}{2}-1} \exp(-\frac{\nu_0 \tau_0^2}{2\sigma^2}) \end{aligned}$$

where $\nu_0 \in \mathbb{R}_+$ is the *prior degrees of freedom* and $\tau_0^2 \in \mathbb{R}_+$ is the *prior mean* of $\sigma_2$. We then place a Gaussian prior over $\boldsymbol{\beta}$:

$$P(\boldsymbol{\beta} \mid \sigma^2, \mathbf{m}_0,\, L_0) = \mathcal{N}(\boldsymbol{\beta};\, \mathbf{m}_0, \sigma^2 L_0^{-1})$$

where $\mathbf{m}_0 \in \mathbb{R}^p$ is the *prior mean* of the coefficients, and $L_0$ is a positive definite $p \times p$ *precision matrix*. We collect all *hyperparameters* into the vector $\boldsymbol{\eta}_0 = (\nu_0, \tau_0^2, \mathbf{m}_0, L_0)$.

Notice that the prior over $\boldsymbol{\beta}$ *depends on* $\sigma^2$. We can equivalently express the joint prior over both parameters as the *normal inverse chi-squared distribution (NIX)*:

$$\begin{aligned} P(\boldsymbol{\beta}, \sigma^2 \mid \boldsymbol{\eta}_0) &= \mathrm{NIX}(\boldsymbol{\beta},\, \sigma^2;\, \mathbf{m}_0, L_0, v_0, \tau_0^2) \\ &= \chi^{-2}(\sigma^2;\, v_0, \tau_0^2)\, \mathcal{N}(\boldsymbol{\beta};\, \mathbf{m}_0, \sigma^2 L_0^{-1}) \end{aligned}$$

The **normal inverse chi-squared (NIX) distribution is a conjugate prior for the likelihood** in equation 1.

# PyTorch

You will use PyTorch to complete the coding portions of this assignment. If you are unfamiliar with PyTorch, this webpage provides an introductory tutorial to PyTorch tensors. Another good resource is homework 0 of STAT 305C, which you could work through for practice.

# Problem 1: Derive the Posterior [Math]

Derive the posterior distribution $p(\boldsymbol{\beta}, \sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0)$ where $\boldsymbol{\eta}_0 = (\nu_0, \tau_0^2, \mathbf{m}_0, L_0)$. Since the NIX distribution is the conjugate prior, the posterior should be of the same form as

the prior (i.e., another NIX distribution):

$$\begin{aligned} p(\boldsymbol{\beta}, \sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0) &= \mathrm{NIX}(\boldsymbol{\beta},\ \sigma^2;\ \mathbf{m}_n, L_n, v_n, \tau_n^2) \\ &= \chi^{-2}(\sigma^2 \mid \nu_n, \tau_n^2)\mathcal{N}(\boldsymbol{\beta} \mid \mathbf{m}_n, \sigma^2 L_n^{-1}) \end{aligned}$$

for some *posterior parameters* $\nu_n$, $\tau_n^2$, $\mathbf{m}_n$, and $L_n$. Your job is to provide the exact form of these parameters.

**Hint 1:** Remember that the "standard procedure" for deriving the posterior distribution is to write down the joint distribution (on both parameters and data), and then only collect the terms involving the parameters to obtain the "kernel" of the posterior. But, in this setting, you have to be very careful to keep both $\boldsymbol{\beta}$ and $\sigma^2$, because we are asking for the *joint posterior*.

**Hint 2:** When working with quadratic forms, a useful operation is to complete the square; for any $\mathbf{a} \in \mathbb{R}^n$, $B \in \mathbb{R}^{n \times n}$, and $\mathbf{c} \in \mathbb{R}^n$:

$$\mathbf{a}^\top B\mathbf{a} - 2\mathbf{a}^\top B\mathbf{c} = (\mathbf{a} - \mathbf{c})^\top B(\mathbf{a} - \mathbf{c}) - \mathbf{c}^\top B\mathbf{c}$$

# Q1 ans

---

Note: I handwrite my solutions and then use ai to convert to latex. I do my best to check but some errors my slip by.

We need to find the parameters of the NIX.

$$p(\boldsymbol{\beta}, \sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0) \propto p(\mathbf{y} \mid \boldsymbol{\beta}, \sigma^2, X) \cdot p(\boldsymbol{\beta} \mid \sigma^2, \mathbf{m}_0, L_0) \cdot p(\sigma^2 \mid \nu_0, \tau_0^2)$$

Each factor written out:

$$p(\mathbf{y} \mid \boldsymbol{\beta}, \sigma^2, X) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_i - \mathbf{x}_i^T\boldsymbol{\beta})^2\right)$$

$$= (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{1}{2\sigma^2}(\mathbf{y} - X\boldsymbol{\beta})^T(\mathbf{y} - X\boldsymbol{\beta})\right)$$

$$p(\boldsymbol{\beta} \mid \sigma^2, \mathbf{m}_0, L_0) = (2\pi)^{-p/2}|\sigma^2 L_0^{-1}|^{-1/2} \exp\left(-\frac{1}{2\sigma^2}(\boldsymbol{\beta} - \mathbf{m}_0)^T L_0(\boldsymbol{\beta} - \mathbf{m}_0)\right)$$

$$p(\sigma^2 \mid \nu_0, \tau_0^2) = \frac{(\frac{\nu_0\tau_0^2}{2})^{\nu_0/2}}{\Gamma(\nu_0/2)}(\sigma^2)^{-\nu_0/2-1} \exp\left(-\frac{\nu_0\tau_0^2}{2\sigma^2}\right)$$

Now we combine and drop constants

$$p(\boldsymbol{\beta}, \sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0) \propto (\sigma^2)^{-n/2-p/2-\nu_0/2-1}.$$

$$\cdot \exp\left(-\frac{1}{2\sigma^2}\left[(\mathbf{y}-X\boldsymbol{\beta})^T(\mathbf{y}-X\boldsymbol{\beta})+(\boldsymbol{\beta}-\mathbf{m}_0)^T L_0(\boldsymbol{\beta}-\mathbf{m}_0)+\nu_0\tau_0^2\right]\right)$$

## Working on $\nu_n$:

from above we need:

$$(\sigma^2)^{-n/2-p/2-\nu_0/2-1}$$

Doing the same for the posterior we get:

$$(\sigma^2)^{-\nu_n/2-1-p/2}$$

then equate:

$$-\frac{\nu_n}{2}-1-\frac{p}{2}=-\frac{n}{2}-\frac{p}{2}-\frac{\nu_0}{2}-1$$

$$-\frac{\nu_n}{2}=-\frac{n}{2}-\frac{\nu_0}{2}$$

$$\nu_n=\nu_0+n$$

## Working on $L_n$ and $m_n$:

from above, we need everything inside the exponent:

$$-\frac{1}{2\sigma^2}\left[(\mathbf{y}-X\boldsymbol{\beta})^T(\mathbf{y}-X\boldsymbol{\beta})+(\boldsymbol{\beta}-\mathbf{m}_0)^T L_0(\boldsymbol{\beta}-\mathbf{m}_0)+\nu_0\tau_0^2\right]$$

Expanding the terms using the identity given in the hint:

$$(\mathbf{y}-X\boldsymbol{\beta})^T(\mathbf{y}-X\boldsymbol{\beta})$$

$$=\mathbf{y}^T\mathbf{y}-2\mathbf{y}^TX\boldsymbol{\beta}+\boldsymbol{\beta}^TX^TX\boldsymbol{\beta}$$

$$=\boldsymbol{\beta}^TX^TX\boldsymbol{\beta}-2\boldsymbol{\beta}^TX^T\mathbf{y}+\mathbf{y}^T\mathbf{y}$$

Repeat with the second term:

$$(\boldsymbol{\beta}-\mathbf{m}_0)^T L_0(\boldsymbol{\beta}-\mathbf{m}_0)$$

$$=\boldsymbol{\beta}^TL_0\boldsymbol{\beta}-2\boldsymbol{\beta}^TL_0\mathbf{m}_0+\mathbf{m}_0^TL_0\mathbf{m}_0$$

$$=\boldsymbol{\beta}^TL_0\boldsymbol{\beta}-2\boldsymbol{\beta}^TL_0\mathbf{m}_0+\mathbf{m}_0^TL_0\mathbf{m}_0$$

and finally:

$$\nu_0\tau_0^2$$

When combining and splitting by arranging by quadratic and linear terms we get:

$$-\frac{1}{2\sigma^2}\left[\boldsymbol{\beta}^T(X^TX+L_0)\boldsymbol{\beta}-2\boldsymbol{\beta}^T(X^T\mathbf{y}+L_0\mathbf{m}_0)+\mathbf{y}^T\mathbf{y}+\mathbf{m}_0^TL_0\mathbf{m}_0+\nu_0\tau_0^2\right]$$

Given that we know the kernel of Gaussian, we just have to match the coefficients across quadratic and linear terms of the kernel:

$$-\frac{1}{2\sigma^2}(\boldsymbol{\beta}-\mathbf{m}_n)^TL_n(\boldsymbol{\beta}-\mathbf{m}_n)$$

or in our case:

$$-\frac{1}{2\sigma^2}\left[\boldsymbol{\beta}^TL_n\boldsymbol{\beta}-2\boldsymbol{\beta}^TL_n\mathbf{m}_n+\mathbf{m}_n^TL_n\mathbf{m}_n\right]$$

We find that:

$$L_n=X^TX+L_0$$

Also:

$$L_n\mathbf{m}_n=X^T\mathbf{y}+L_0\mathbf{m}_0$$

$$\mathbf{m}_n=L_n^{-1}(X^T\mathbf{y}+L_0\mathbf{m}_0)$$

$$\mathbf{m}_n=(X^TX+L_0)^{-1}(X^T\mathbf{y}+L_0\mathbf{m}_0)$$

## Working on $\tau_n^2$:

recall from the previous step:

$$\boldsymbol{\beta}^TL_n\boldsymbol{\beta}-2\boldsymbol{\beta}^TL_n\mathbf{m}_n=(\boldsymbol{\beta}-\mathbf{m}_n)^TL_n(\boldsymbol{\beta}-\mathbf{m}_n)-\mathbf{m}_n^TL_n\mathbf{m}_n$$

putting the full exponent together:

$$-\frac{1}{2\sigma^2}\left[\boldsymbol{\beta}^TL_n\boldsymbol{\beta}-2\boldsymbol{\beta}^TL_n\mathbf{m}_n+\mathbf{y}^T\mathbf{y}+\mathbf{m}_0^TL_0\mathbf{m}_0+\nu_0\tau_0^2\right]$$

substituting the completed square:

$$=-\frac{1}{2\sigma^2}\left[(\boldsymbol{\beta}-\mathbf{m}_n)^TL_n(\boldsymbol{\beta}-\mathbf{m}_n)-\mathbf{m}_n^TL_n\mathbf{m}_n+\mathbf{y}^T\mathbf{y}+\mathbf{m}_0^TL_0\mathbf{m}_0+\nu_0\tau_0^2\right]$$

Now back to part 1 with $(\sigma^2)$, we can put the full posterior together:

$$p(\boldsymbol{\beta},\sigma^2\mid\cdots)\propto(\sigma^2)^{-\nu_n/2-1-p/2}\exp\left(-\frac{1}{2\sigma^2}\left[\text{... from above...}\right]\right)$$

We then split this up as we only want to the terms relevant to $\chi^{-2}$ density, so what interests us is this part: (other part goes into the gaussian)

$$(\sigma^2)^{-\nu_n/2-1} \exp\left(-\frac{\left(\nu_0\tau_0^2 + \mathbf{y}^T\mathbf{y} + \mathbf{m}_0^T L_0 \mathbf{m}_0 - \mathbf{m}_n^T L_n \mathbf{m}_n\right)}{2\sigma^2}\right)$$

of:

$$\chi^{-2}(\sigma^2 \mid \nu_n,\, \tau_n^2)$$

This matches the exponent of $\chi^{-2}$ so we can isolate:

$$\nu_n\tau_n^2 = \nu_0\tau_0^2 + \mathbf{y}^T\mathbf{y} + \mathbf{m}_0^T L_0 \mathbf{m}_0 - \mathbf{m}_n^T L_n \mathbf{m}_n$$

$$\tau_n^2 = \frac{1}{\nu_n}\left(\nu_0\tau_0^2 + \mathbf{y}^T\mathbf{y} + \mathbf{m}_0^T L_0 \mathbf{m}_0 - \mathbf{m}_n^T L_n \mathbf{m}_n\right)$$

or

$$\tau_n^2 = \frac{1}{\nu_0 + n}\left(\nu_0\tau_0^2 + \mathbf{y}^T\mathbf{y} + \mathbf{m}_0^T L_0 \mathbf{m}_0 - \mathbf{m}_n^T L_n \mathbf{m}_n\right)$$

## Finally:

The posterior is:

$$p(\boldsymbol{\beta}, \sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0) = \mathrm{NIX}(\boldsymbol{\beta}, \sigma^2;\ \mathbf{m}_n, L_n, \nu_n, \tau_n^2)$$

where:

$$\nu_n = \nu_0 + n$$

$$L_n = L_0 + X^T X$$

$$\mathbf{m}_n = L_n^{-1}(L_0\mathbf{m}_0 + X^T\mathbf{y})$$

$$\tau_n^2 = \frac{1}{\nu_n}\left(\nu_0\tau_0^2 + \mathbf{y}^T\mathbf{y} + \mathbf{m}_0^T L_0 \mathbf{m}_0 - \mathbf{m}_n^T L_n \mathbf{m}_n\right)$$

---

# Problem 2: The Posterior Mean [Math]

a. What does the posterior mean $\mathbb{E}[\boldsymbol{\beta} \mid \mathbf{y}, X, \boldsymbol{\eta}_0]$ equal in the uninformative limit where $L_0 \to 0$ and $\nu_0 \to 0$?

b. What does the posterior mean $\mathbb{E}[\sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0]$ equal in the uninformative limit where $L_0 \to 0$ and $\nu_0 \to 0$? Write your answer in terms of the *hat matrix* $\mathbf{H} = X(X^\top X)^{-1}X^\top$.

---

# Part a

from problem 1 we know the posterior on $\beta$ (conditional on $\sigma^2$) is Gaussian with mean $M_n$. Since the mean of a Gaussian is also its expectation we can write:

$$\mathbb{E}[\boldsymbol{\beta} \mid \mathbf{y}, X, \boldsymbol{\eta}_0] = \mathbf{m}_n = L_n^{-1}(L_0\mathbf{m}_0 + X^T\mathbf{y})$$

We must note that the mean does not depend on $\sigma^2$:

$$\mathbb{E}[\boldsymbol{\beta} \mid \mathbf{y}, X] = \mathbb{E}_{\sigma^2}[\mathbf{m}_n] = \mathbf{m}_n$$

Now take the limit. As $L_0$ to 0:

$$L_n = L_0 + X^T X$$

goes to:

$$X^T X$$

and

$$L_n\mathbf{m}_n = L_0\mathbf{m}_0 + X^T\mathbf{y}$$

goes to:

$$X^T\mathbf{y}$$

Therefore:

$$\mathbf{m}_n => (X^T X)^{-1} X^T\mathbf{y}$$

and finally:

$$\lim_{L_0->0,\ \nu_0->0} \mathbb{E}[\boldsymbol{\beta} \mid \mathbf{y}, X, \boldsymbol{\eta}_0] = (X^T X)^{-1} X^T\mathbf{y}$$

# Part b

From q1 the posterior marginal on $\sigma^2$ is $\chi^{-2}(\nu_n, \tau_n^2)$. The mean of a inverse chi-squared distribution is:

$$\mathbb{E}[\sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0] = \frac{\nu_n}{\nu_n - 2}\,\tau_n^2$$

Finding the limit of $\nu_n$ as $v_0 - > 0$ :

$$\nu_n = \nu_0 + n - > n$$

Now for $\tau_n^2$. From q1 we know:

$$\tau_n^2 = \frac{1}{\nu_n}\left(\nu_0\tau_0^2 + \mathbf{y}^T\mathbf{y} + \mathbf{m}_0^T L_0\mathbf{m}_0 - \mathbf{m}_n^T L_n\mathbf{m}_n\right)$$

As $L_0 -> 0$ and $\nu_0 -> 0$:

1. $\nu_0 \tau_0^2 -> 0$
2. $\mathbf{m}_0^T L_0 \mathbf{m}_0 -> 0$
3. $L_n -> X^T X$
4. $\mathbf{m}_n -> (X^T X)^{-1} X^T \mathbf{y}$
5. $\nu_n -> n$

so we get:

$$\tau_n^2 -> \frac{1}{n}\Big(\mathbf{y}^T\mathbf{y} - [*]\Big)$$

where $*$ is:

$$\begin{aligned}
&\Big[(X^TX)^{-1}X^T\mathbf{y}\Big]^T X^TX\Big[(X^TX)^{-1}X^T\mathbf{y}\Big] \\
&= \mathbf{y}^T X\underbrace{(X^TX)^{-1}X^TX}_{=\,I}(X^TX)^{-1}X^T\mathbf{y} \\
&= \mathbf{y}^T \underbrace{X(X^TX)^{-1}X^T}_{=\,H}\mathbf{y} \\
&= \mathbf{y}^T H\mathbf{y}
\end{aligned}$$

putting it together:

$$\tau_n^2 -> \frac{1}{n}\Big(\mathbf{y}^T\mathbf{y} - \mathbf{y}^T H\mathbf{y}\Big) = \frac{\mathbf{y}^T(I_n - H)\mathbf{y}}{n}$$

and computing the expectation:

$$\mathbb{E}[\sigma^2] = \frac{\nu_n}{\nu_n - 2}\,\tau_n^2 -> \frac{n}{n-2}\cdot\frac{\mathbf{y}^T(I_n - H)\mathbf{y}}{n}$$

$$\lim_{L_0 -> 0,\ \nu_0 -> 0} \mathbb{E}[\sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0] = \frac{\mathbf{y}^T(I_n - H)\mathbf{y}}{n-2}$$

---

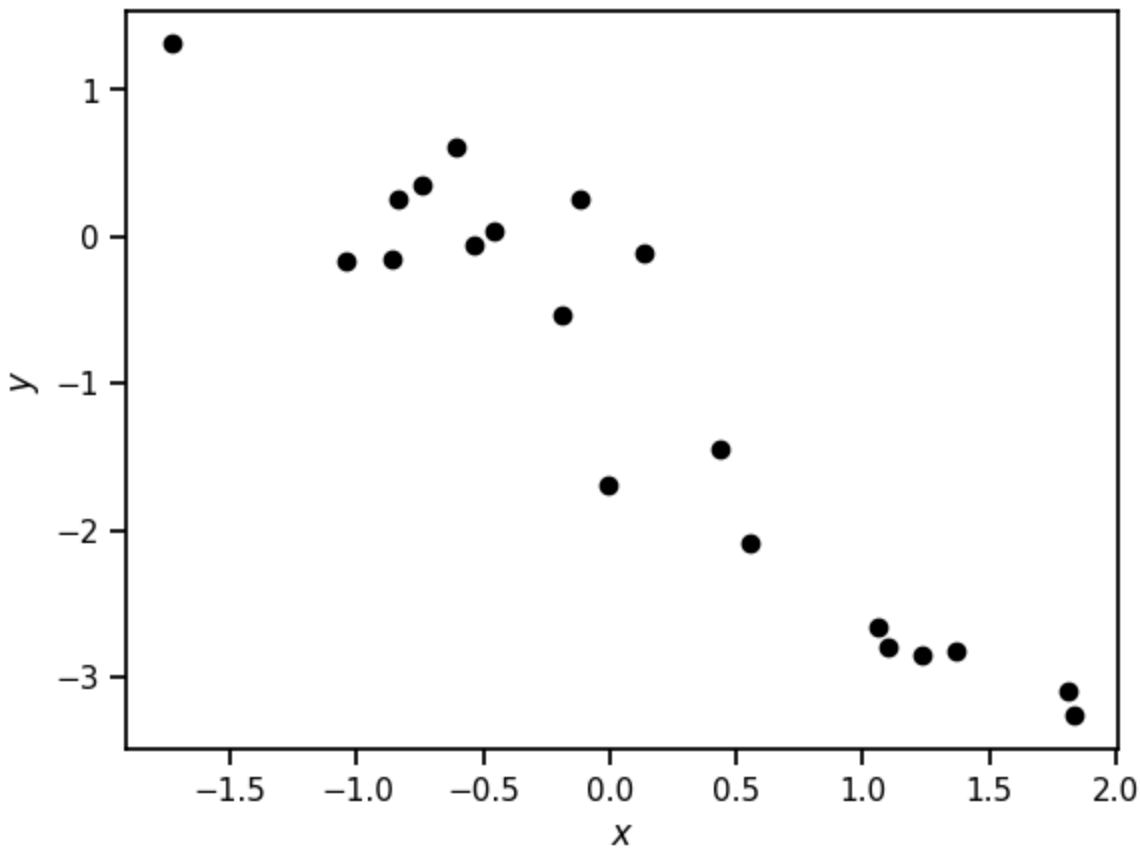
# Synthetic Data

We'll do some simple analysis of a synthetic dataset with $n = 20$ data points. Each data point has covariates $\mathbf{x}_i = (1, x_i) \in \mathbb{R}^2$ and scalar outcomes $y_i \in \mathbb{R}$. It looks like this:

In [3]:
```
# Download the data (uncomment the line below)
# !wget https://github.com/aschein/stat_348_2026/raw/main/assignments/hw1.pt

# Load the data.
```

```
# X = [[1, x_1]
#      [1, x_2]
#         ...
#      [1, x_N]]
#
# y = [y_1, ..., y_N]
X, y = torch.load("hw1.pt")

plt.plot(X[:, 1], y, 'ko')
plt.xlabel("$x$")
plt.ylabel("$y$")
```

Out[3]: Text(0, 0.5, '$y$')




Here, the outcomes were simulated from a linear regression with Gaussian noise according to some true parameters (not given). You will compute and visualize the posterior distribution over the weights and variance given the data.

## Problem 3: Compute the posterior [Code]

Write a function to compute the posterior parameters given data and hyperparameters.

*Hints*: You may find the following commands in PyTorch useful:

- If `a` is a tensor, `a.shape` is a tuple containing the shape of `a`.
- If `a` is a tensor, `a.T` returns the transpose of `a`.

- `torch.linalg.solve`
- `*` denotes element-wise multiplication while `@` denotes standard matrix-matrix or matrix-vector multiplication.

In [6]:
```
def compute_posterior(X, y, nu_0, tau_0, m_0, L_0):
    """
    Compute the posterior parameters nu_n, tau_n, m_n, and L_n
    given covariates X, outcomes y, and hyperparameters.

    Args:
        X:          (n, p) tensor of covariates
        y:          (n,) tensor of outcomes
        nu_0:       prior degrees of freedom
        tau_0:  prior mean of the variance parameter
        m_0:       prior mean of the weights
        L_0:   prior precision of the weights

    Returns:
        nu_n:       posterior degrees of freedom
        tau_n:  posterior scale of the variance parameter
        m_n:       posterior mean of the weights
        L_n:   posterior precision of the weights
    """

    n = X.shape[0]

    # Posterior d.o.f from q1 : param 1
    nu_n = nu_0 + n

    # Posterior precision : param 2
    L_n = L_0 + X.T @ X

    # Posterior mean : param 3
    m_n = torch.linalg.solve(L_n, X.T @ y + L_0 @ m_0)

    # Posterior scale : param 4
    nu_n_tau_n_sq = (nu_0 * tau_0**2      + y @ y      + m_0 @ L_0 @ m_0

    tau_n = torch.sqrt(nu_n_tau_n_sq / nu_n)



    return nu_n, tau_n, m_n, L_n
```

Please run the following code to print your answers:

In [7]:
```
# Test:
hyperparams = dict(
    nu_0=torch.tensor(1.0),
    tau_0=torch.tensor(1.0),
    m_0=torch.zeros(2),
    L_0=0.1 * torch.eye(2)
)
```

```
nu_n, tau_n, m_n, L_n = compute_posterior(X, y, **hyperparams)
print("nu_n:       \n", nu_n)
print("")
print("tau_n:  \n", tau_n)
print("")
print("m_n:       \n", m_n)
print("")
print("L_n:   \n", L_n)
```

```
nu_n:
 tensor(21.)

tau_n:
 tensor(0.5228)

m_n:
 tensor([-0.8777, -1.3646])

L_n:
 tensor([[20.1000,  2.4126],
        [ 2.4126, 20.0170]])
```

## Problem 4: Plot the posterior density of the variance [Code]

Plot $p(\sigma^2 \mid X, \mathbf{y}, \boldsymbol{\eta}_0)$ vs $\sigma^2$ over the interval $[10^{-3}, 2]$, where $X$ and $\mathbf{y}$ continue to be the synthetic data we downloaded and used in Problem 3.

You may use the `ScaledInvChiSq` distribution object below, which we copied from the demo for Lecture 1.

*Hint*: In Python, you can use `dir(object)` to list the attributes and functions that an object supports.

*Hint*: To learn more about PyTorch distributions, see the docs.

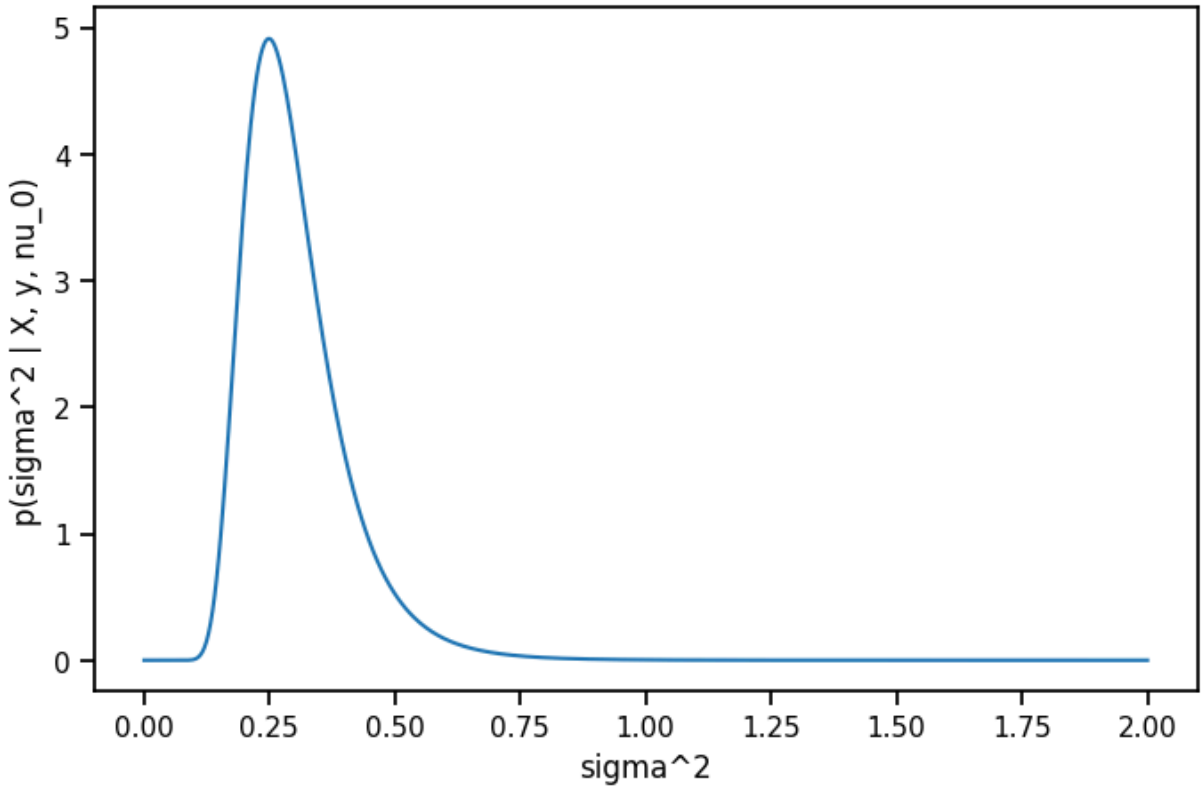
In [9]:
```
class ScaledInvChiSq(TransformedDistribution):

    def __init__(self, dof, scale):
        """
        Implementation of the scaled inverse \chi^2 distribution,

        ..math:
            \chi^{-2}(\nu_0, \tau_0^2)

        It is equivalent to an inverse gamma distribution, which we implemer
        as a transformation of a Gamma distribution. Thus, this class inheri
        functions like `log_prob` from its parent.

        Args:
            dof:   degrees of freedom parameter
            scale: scale of the $\chi^{-2}$ distribution.
        """
```

```
        base = Gamma(dof / 2, dof * scale / 2)
        transforms = [PowerTransform(-1)]
        TransformedDistribution.__init__(self, base, transforms)
        self.dof = dof
        self.scale = scale
```

In [15]:
```
# print(dir(ScaledInvChiSq(1.0, 1.0)))
```

```
['__annotations__', '__class__', '__delattr__', '__dict__', '__dir__', '__do
c__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getstate__',
'__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__',
'__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr_
_', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref_
_', '_batch_shape', '_event_shape', '_extended_shape', '_get_checked_instanc
e', '_monotonize_cdf', '_validate_args', '_validate_sample', 'arg_constraint
s', 'base_dist', 'batch_shape', 'cdf', 'dof', 'entropy', 'enumerate_suppor
t', 'event_shape', 'expand', 'has_enumerate_support', 'has_rsample', 'icdf',
'log_prob', 'mean', 'mode', 'perplexity', 'rsample', 'sample', 'sample_n',
'scale', 'set_default_validate_args', 'stddev', 'support', 'transforms', 'va
riance']
```

In [52]:
```
sigma2_grid = torch.linspace(1e-3, 2, 500)

# inverse chi-squared distribution
posterior_sigma2 = ScaledInvChiSq(dof=nu_n, scale=tau_n**2)

log_probs = posterior_sigma2.log_prob(sigma2_grid)
probs = torch.exp(log_probs)

plt.figure(figsize=(8, 5))
plt.plot(sigma2_grid, probs)
plt.xlabel("sigma^2")
plt.ylabel("p(sigma^2 | X, y, nu_0)")
plt.show()
```




## Problem 5: Plot posterior samples of the regression function. [Code]

Draw 50 samples from the posterior marginal distribution over the weights $\boldsymbol{\beta} \in \mathbb{R}^2$. For each sample, compute the expected value of $y$ on a grid of points $x$ evenly spaced between $[-3, 3]$. Remember that our covariates were defined as $\mathbf{x} = (1, x)$ so that for each sample of the weights you get a line for $\mathbb{E}[y \mid x, \boldsymbol{\beta}]$ as a function of $x$. Plot these 50 lines on top of each other to get a sense of the posterior uncertainty in the regression function. (You may want to plot each line with some transparency, like `alpha=0.1`.) Overlay the observed data points.

*Hint*: You may find `torch.inverse` useful.

*Hint*: Remember that in the generative model we have posited, the distribution of $\boldsymbol{\beta}$ depends on $\sigma^2$.

In [17]:
```
posterior_sigma2 = ScaledInvChiSq(dof=nu_n, scale=tau_n**2)
sigma2_samples = posterior_sigma2.sample((50,))
sigma2_samples
```

Out[17]:
```
tensor([0.1871, 0.2962, 0.1550, 0.2963, 0.1978, 0.2471, 0.3624, 0.4107, 0.4
940,
        0.3516, 0.4466, 0.3657, 0.4609, 0.6193, 0.2214, 0.2726, 0.2510, 0.2
255,
        0.5120, 0.2042, 0.2261, 0.2528, 0.4425, 0.5818, 0.4747, 0.1794, 0.2
418,
        0.3502, 0.2919, 0.2752, 0.4269, 0.2118, 0.3304, 0.2419, 0.5989, 0.1
780,
        0.2066, 0.2102, 0.4183, 0.2008, 0.1191, 0.2180, 0.3766, 0.1908, 0.3
432,
        0.3067, 0.2705, 0.3537, 0.5303, 0.2468])
```

In [28]:
```python
L_n_inv = torch.inverse(L_n)
beta_samples = []
for s2 in sigma2_samples:
    cov = s2 * L_n_inv
    dist = MultivariateNormal(m_n, covariance_matrix=cov)
    beta_samples.append(dist.sample())
beta_samples = torch.stack(beta_samples)

print(beta_samples.shape)
beta_samples[:5]
```

```
torch.Size([50, 2])
```

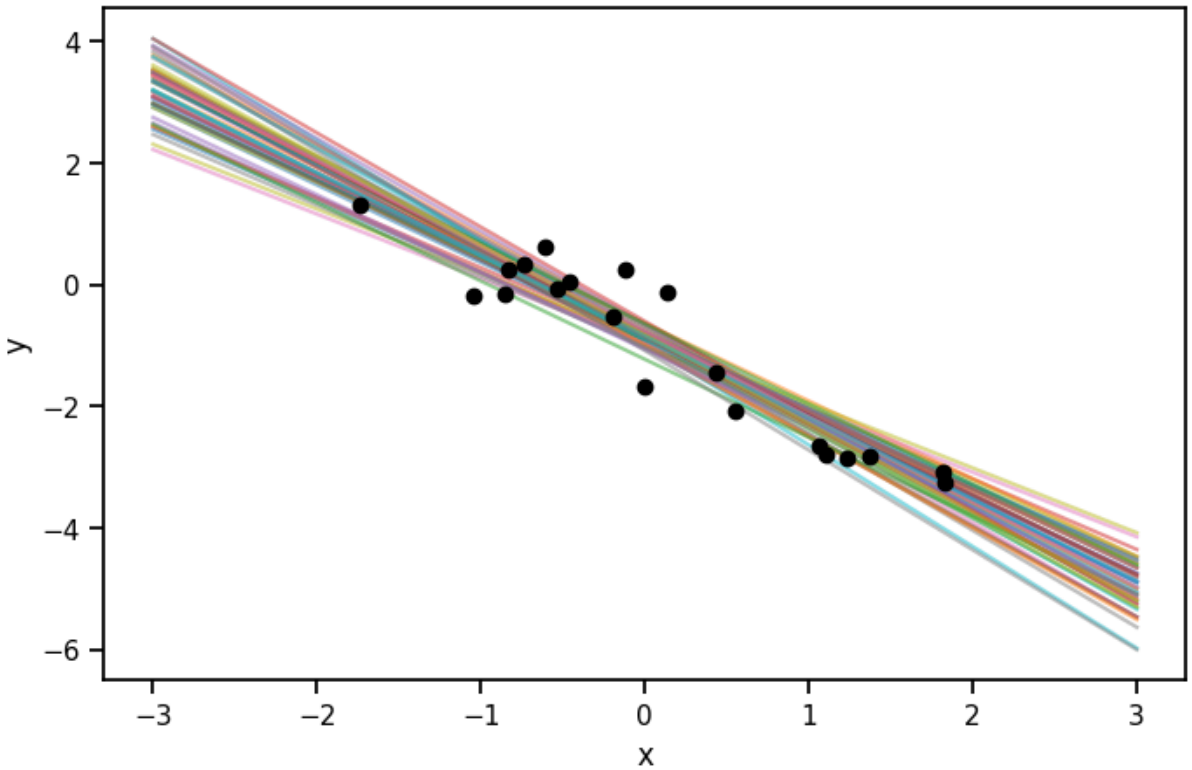
Out[28]:
```
tensor([[-0.9135, -1.3204],
        [-0.6599, -1.4808],
        [-0.9100, -1.4302],
        [-0.8793, -1.1588],
        [-0.9030, -1.3289]])
```

In [34]:
```python
x_grid = torch.linspace(-3, 3, 200)
X_grid = torch.stack([torch.ones_like(x_grid), x_grid], dim=1)  # (200, 2)

print(x_grid.shape)
print(X_grid.shape)

# x_grid[:5]
X_grid[:5]
```

```
torch.Size([200])
torch.Size([200, 2])
```

Out[34]:
```
tensor([[ 1.0000, -3.0000],
        [ 1.0000, -2.9698],
        [ 1.0000, -2.9397],
        [ 1.0000, -2.9095],
        [ 1.0000, -2.8794]])
```

In [53]:
```
y_lines = X_grid @ beta_samples.T

plt.figure(figsize=(8, 5))
for i in range(50):
    plt.plot(x_grid, y_lines[:, i], alpha=0.5)
plt.plot(X[:, 1], y, 'ko')
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```




## Problem 6: Posterior Predictive Distribution [Math]

The subparts of this problem will walk you through deriving the posterior predictive distribution of the outcome at a new input $\mathbf{x}_{n+1}$. That is, computing,

$$p(y_{n+1} \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0)$$

integrating over the posterior distribution on the coefficients $\boldsymbol{\beta}$ and variance $\sigma^2$. Remember that you found this posterior distribution in Problem 1, but for the purpose of this question it's enough to leave it in the form

$$p(\boldsymbol{\beta}, \sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0) = \chi^{-2}(\sigma^2 \mid \nu_n, \tau_n^2)\,\mathcal{N}(\boldsymbol{\beta} \mid \mathbf{m}_n, \sigma^2 L_n^{-1}),$$

i.e. you don't need to plug in the values for for some $\nu_n$, $\tau_n^2$, $\mathbf{m}_n$, and $L_n$ that you found in Problem 1.

### Problem 6a

Using the product rule of probability, write out the joint distribution of the posterior over the parameters and the observation of the next data point $y_{n+1}$:

$$p(y_{n+1}, \boldsymbol{\beta}, \sigma^2 \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0).$$

You can (and please do) replace any densities from a known family with the notation $\text{symbol for the family}(\text{variable name} \mid \text{parameters})$. (We follow this notation in how we write out the posterior above).

---

By product rule we have:

$$p(y_{n+1}, \boldsymbol{\beta}, \sigma^2 \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0) = \underbrace{p(y_{n+1} \mid \boldsymbol{\beta}, \sigma^2, \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0)}_{\text{(i)}} \cdot \underbrace{p(\boldsymbol{\beta}, \sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0)}_{\text{(ii)}}$$

looking at the first factor we note that once we know the parameters $\boldsymbol{\beta}$ and $\sigma^2$, the new observation $y_{n+1}$ only depends on $\mathbf{x}_{n+1}$ as it is conditionally independent of the old data $(\mathbf{y}, X)$ and hyperparameters $\boldsymbol{\eta}_0$.

this then becomes:

$$p(y_{n+1} \mid \boldsymbol{\beta}, \sigma^2, \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0) = p(y_{n+1} \mid \boldsymbol{\beta}, \sigma^2, \mathbf{x}_{n+1})$$

The second factor is the posterior from problem 1:

$$p(\boldsymbol{\beta}, \sigma^2 \mid \mathbf{y}, X, \boldsymbol{\eta}_0) = \chi^{-2}(\sigma^2 \mid \nu_n, \tau_n^2)\, \mathcal{N}(\boldsymbol{\beta} \mid \mathbf{m}_n,\, \sigma^2 L_n^{-1})$$

so when combined:

$$p(y_{n+1}, \boldsymbol{\beta}, \sigma^2 \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0) = \mathcal{N}(y_{n+1} \mid \mathbf{x}_{n+1}^T \boldsymbol{\beta},\, \sigma^2) \cdot \chi^{-2}(\sigma^2 \mid \nu_n, \tau_n^2) \cdot$$
$$\cdot \mathcal{N}(\boldsymbol{\beta} \mid \mathbf{m}_n,\, \sigma^2 L_n^{-1})$$

---

## Problem 6b

Now using the sum rule of probability, compute the posterior predictive distribution

$$p(y_{n+1}, | \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0) = \int p(y_{n+1}, \boldsymbol{\beta}, \sigma^2 \mid \boldsymbol{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0)\, d\boldsymbol{\beta}\, d\sigma^2.$$

*Hint:* You can do this integral without taking any integrals! Think about conjugate families and how the Student's T distribution arises (e.g., see these slides).

---

compute :

$$p(y_{n+1} \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0) = \iint p(y_{n+1}, \boldsymbol{\beta}, \sigma^2 \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0)\, d\boldsymbol{\beta}\, d\sigma^2$$

we can do this in 2 parts:

## part 1:

Integrate out $\boldsymbol{\beta}$ (given $\sigma^2$).

Plugging in the three factors from 6a, the inner integral is: (ommitting $\chi^{-2}$ as it does not depend on $\beta$)

$$\int p(y_{n+1}, \boldsymbol{\beta}, \sigma^2 \mid -)\, d\boldsymbol{\beta}$$

$$= \chi^{-2}(\sigma^2 \mid \nu_n, \tau_n^2) \int \mathcal{N}(y_{n+1} \mid \mathbf{x}_{n+1}^T \boldsymbol{\beta}, \sigma^2)\, \mathcal{N}(\boldsymbol{\beta} \mid \mathbf{m}_n, \sigma^2 L_n^{-1})\, d\boldsymbol{\beta}$$

Recall from lecture:

If $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ and $y = A\mathbf{z} + b + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, R)$ is independent, then $y$ is also Gaussian with:

1. Mean: $A\boldsymbol{\mu} + b$
2. Variance: $A\Sigma A^T + R$

Finding these yields:

Mean of $y_{n+1} \mid \sigma^2$:

$$\mathbb{E}[y_{n+1} \mid \sigma^2]$$

$$= \mathbf{x}_{n+1}^T \cdot \mathbf{m}_n + 0$$

$$= \mathbf{x}_{n+1}^T \mathbf{m}_n$$

Variance of $y_{n+1} \mid \sigma^2$:

$$\mathrm{Var}[y_{n+1} \mid \sigma^2]$$

$$= \mathrm{Var}[\mathbf{x}_{n+1}^T \boldsymbol{\beta} \mid \sigma^2] + \mathrm{Var}[\varepsilon_{n+1}]$$

looking at the first term and using $\mathrm{Var}[A\mathbf{z}] = A\,\mathrm{Cov}(\mathbf{z})\,A^T$:

$$\mathrm{Var}[\mathbf{x}_{n+1}^T \boldsymbol{\beta} \mid \sigma^2]$$

$$= \mathbf{x}_{n+1}^T (\sigma^2 L_n^{-1}) \mathbf{x}_{n+1}$$

$$= \sigma^2 \cdot \mathbf{x}_{n+1}^T L_n^{-1} \mathbf{x}_{n+1}$$

The second:

$$\begin{aligned}
&\mathrm{Var}[y_{n+1} \mid \sigma^2] \\
&= \sigma^2 \cdot \mathbf{x}_{n+1}^T L_n^{-1} \mathbf{x}_{n+1} + \sigma^2 \\
&= \sigma^2 \big(1 + \mathbf{x}_{n+1}^T L_n^{-1} \mathbf{x}_{n+1}\big)
\end{aligned}$$

Therefore after integrating out $\beta$ we get the distrivution with the parameters:

$$y_{n+1} \mid \sigma^2, \mathbf{y}, X \sim \mathcal{N}\big(\mathbf{x}_{n+1}^T \mathbf{m}_n, \;\; \sigma^2 \big(1 + \mathbf{x}_{n+1}^T L_n^{-1} \mathbf{x}_{n+1}\big)\big)$$

## part 2:

we now need to compute:

$$p(y_{n+1} \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0) =$$

$$= \int \underbrace{\mathcal{N}(y_{n+1} \mid \mathbf{x}_{n+1}^T \mathbf{m}_n, \; \sigma^2 \big(1 + \mathbf{x}_{n+1}^T L_n^{-1} \mathbf{x}_{n+1}\big)}_{\text{from part 1}} \cdot \underbrace{\chi^{-2}(\sigma^2 \mid \nu_n, \tau_n^2)}_{\text{posterior on } \sigma^2} \, d\sigma^2$$

This is a Gaussian whose variance is proportional to $\sigma^2$.

We can now use the setup from the slides to directly map to our problem.

There, for the scalar case with:

1. mean: $\mu \mid \sigma^2 \sim \mathcal{N}(\mu_N, \sigma^2/\kappa_N$
2. var: $\sigma^2 \sim \chi^{-2}(\nu_N, \sigma_N^2)$

the marginal (integrating out $\sigma^2$) is shown to be: (eq. 43)

$$p(\mu \mid X, \boldsymbol{\eta}) = \mathrm{St}(\mu; \; \nu_N, \; \mu_N, \; \sigma_N^2/\kappa_N)$$

so:

$$p(y_{n+1} \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0) = \mathrm{St}\big(y_{n+1}; \;\; \nu_n, \;\; \text{mean}, \;\; \tau_n^2 (1 + \mathbf{x}_{n+1}^T L_n^{-1} \mathbf{x}_{n+1})\big)$$

replacing the mean:

$$p(y_{n+1} \mid \mathbf{x}_{n+1}, \mathbf{y}, X, \boldsymbol{\eta}_0) = \mathrm{St}\big(y_{n+1}; \;\; \nu_n, \;\; \mathbf{x}_{n+1}^T \mathbf{m}_n, \;\; \tau_n^2 \big(1 + \mathbf{x}_{n+1}^T L_n^{-1} \mathbf{x}_{n+1}\big)\big)$$

note: "St" here refers to the specific marginal distribution mentioned and derived in the slides and whos density funciton is in eq. 44.

---

# Submission Instructions

**Formatting:** check that your code does not exceed 80 characters in line width. You can set *Tools → Settings → Editor → Vertical ruler column* to 80 to see when you've exceeded the limit.

Download your notebook in .ipynb format and use the following commands to convert it to PDF. First, rename your file `<yourlastname>_hw1.ipynb`. Then run:

```
jupyter nbconvert --to webpdf <yourlastname>_hw1.ipynb
```

This uses a headless browser to render the notebook as a PDF (no LaTeX installation required). Math is rendered via MathJax, so it should look the same as in the notebook.

**Installing dependencies:**

```
pip install "nbconvert[webpdf]"
playwright install chromium
```

Getting this HW into PDF form isn't meant to be a burden. If the above doesn't work, one quick and easy approach is to open it as a Jupyter notebook, print, save to pdf. Just make sure your latex math answers aren't cut off so we can grade them.

Please post on Ed or come to OH if there are any other problems submitting the HW.

**Upload** your .pdf file to Gradescope. Please tag your questions!